



程序设计原理

函数2

任课教师：喻 梅
于瑞国
王建荣
李雪威
赵满坤



Overloading Functions

```
/** Return the max between two int values */  
int max(int num1, int num2)  
{  
    if (num1 > num2)  
        return num1;  
    else  
        return num2;  
}
```



Overloading Functions

The max function that was used earlier works only with the int data type. But what if you need to find which of two floating-point numbers has the maximum value? The solution is to create another function with the same name but different parameters, as shown in the following code. Overloading functions enable you to define the function with the same name as long as their signatures are different.



Overloading Functions

```
#include <iostream>
using namespace std;

/** Return the max between two int values */
int max(int num1, int num2)
{
    if (num1 > num2)
        return num1;
    else
        return num2;
}

/** Return the max among three double values */
double max(double num1, double num2, double num3)
{
    return max(max(num1, num2), num3);
}
```

```
/** Find the max between two double values */
double max(double num1, double num2){
    if (num1 > num2)
        return num1;
    else
        return num2;
}

int main(){
    // Invoke the max function with int parameters
    cout << "The maximum between 3 and 4 is " <<
    max(3, 4) << endl;

    // Invoke the max function with the double parameters
    cout << "The maximum between 3.0 and 5.4 is "
    << max(3.0, 5.4) << endl;

    // Invoke the max function with three double parameters
    cout << "The maximum between 3.0, 5.4, and 10.14
    is "
    << max(3.0, 5.4, 10.14) << endl;
    return 0;
}
```



Ambiguous Invocation

Sometimes there may be two or more possible matches for an invocation of a function, but the compiler cannot determine the most specific match. This is referred to as *ambiguous invocation*.

Ambiguous invocation is a compilation error.

```
#include <iostream>
using namespace std;
int maxNumber(int num1, double num2)
{
    if (num1 > num2)
        return num1;
    else
        return num2;
}
double maxNumber(double num1, int num2)
{
    if (num1 > num2)
        return num1;
    else
        return num2;
}
int main()
{
    cout << maxNumber(1, 2) << endl;
    return 0;
}
```



Function Prototypes

Before a function is called, it must be declared first. One way to ensure it is to place the declaration before all function calls. Another way to approach it is to declare a function prototype before the function is called. A function prototype is a function declaration without implementation. The implementation can be given later in the program.



Function Prototypes

```
#include <iostream>
using namespace std;

// Function prototype
int max(int , int);
double max(double num1, double num2);
double max(double num1, double num2, double
num3);

int main()
{
    // Invoke the max function with int parameters
    cout << "The maximum between 3 and 4 is " <<
        max(3, 4) << endl;

    // Invoke the max function with the double
parameters
    cout << "The maximum between 3.0 and 5.4 is "
        << max(3.0, 5.4) << endl;

    // Invoke the max function with three double
parameters
    cout << "The maximum between 3.0, 5.4, and
10.14 is "
        << max(3.0, 5.4, 10.14) << endl;
    return 0;
}
```

```
/** Return the max between two int
values */
int max(int num1, int num2)
{
    if (num1 > num2)
        return num1;
    else
        return num2;
}

/** Find the max between two double
values */
double max(double num1, double num2)
{
    if (num1 > num2)
        return num1;
    else
        return num2;
}

/** Return the max among three
double values */
double max(double num1, double num2,
double num3)
{
    return max(max(num1, num2), num3);
}
```



Default Arguments

C++ allows you to declare functions with default argument values. The default values are passed to the parameters when a function is invoked without the arguments.

```
#include <iostream>
using namespace std;

/** Swap two variables */
void printArea(double radius = 1)
{
    double area = radius * radius * 3.14159;
    cout << "area is " << area << endl;
}

int main()
{
    printArea();
    printArea(4);
    return 0;
}
```




Default Arguments

When a function contains a mixture of parameters with and without default values, those with default values must be declared last.

```
void t1(int x, int y = 0, int z);//illegal
```

```
void t2(int x = 0, int y = 0, int z);//illegal
```

```
void t3(int x, int y = 0, int z = 0);//legal
```

```
void t4(int x = 0, int y = 0, int z = 0);//legal
```



Scope of Variables

A local variable: a variable defined inside a Function.

Scope: the part of the program where the variable can be referenced.

The scope of a variable starts from its declaration and continues to the end of the block that contains the variable. A local variable must be declared before it can be used.



Scope of Local Variables, cont.

You can declare a local variable with the same name multiple times in different blocks in a function, but you cannot declare a local variable twice in the same block.



Global Variables

C++ also allows you to use *global variables*. They are declared outside all functions and are accessible to all functions in its scope. Local variables do not have default values, but global variables are defaulted to zero.

```
#include <iostream>
using namespace std;

void t1(); // function prototype
void t2(); // function prototype

int main()
{
    //cout<<y<<endl;
    t1();
    t2();
    return 0;
}
```

```
int y; // Global variable, default to 0

void t1()
{
    int x = 1;
    cout << "x is " << x << endl;
    cout << "y is " << y << endl;
    x++;
    y++;
    {
        int y=22;
        cout<<::y<<endl;
    }
}

void t2()
{
    int x = 1;
    cout << "x is " << x << endl;
    cout << "y is " << y << endl;
}
```



Unary Scope Resolution

If a local variable name is the same as a global variable name, you can access the global variable using ::globalVariable. The :: operator is known as the *unary scope resolution*. For example, the following code:

```
#include <iostream>
using namespace std;
int v1 = 10;
int main()
{
    int v1 = 5;
    cout << "local variable v1 is " << v1 << endl;
    cout << "global variable v1 is " << ::v1 << endl;
    return 0;
}
```



Scope of Local Variables in a for loop

A variable declared in the initial action part of a for loop header has its scope in the entire loop. But a variable declared inside a for loop body has its scope limited in the loop body from its declaration and to the end of the block that contains the variable.

```
public static void method1() {  
    .  
    .  
    for (int i = 1; i < 10; i++) {  
        .  
        .  
        int j;  
        .  
        .  
    }  
}
```

The scope of i →

The scope of j →



Scope of Local Variables in a for loop

It is fine to declare `i` in two non-nesting blocks

```
void function1()
{
    int x = 1;
    int y = 1;

    for (int i = 1; i < 10; i++)
    {
        x += i;
    }

    for (int i = 1; i < 10; i++)
    {
        y += i;
    }
}
```

It is not a good practice to declare `i` in two nesting blocks

```
void function2()
{
    int i = 1;
    int sum = 0;

    for (int i = 1; i < 10; i++)
    {
        sum += i;
    }

    cout << i << endl;
    cout << sum << endl;
}
```



Static Local Variables

After a function completes its execution, all its local variables are destroyed. Sometimes, it is desirable to retain the value stored in local variables so that they can be used in the next call. C++ allows you to declare static local variables. Static local variables are permanently allocated in the memory for the lifetime of the program. To declare a static variable, use the keyword static.



Static Local Variables

```
#include <iostream>
using namespace std;

void t1(); // function
prototype

int main()
{
    t1();
    t1();
    return 0;
}

void t1()
{
    static int x = 1;
    int y = 1;
    x++;
    y++;
    cout << "x is " << x << endl;
    cout << "y is " << y << endl;
}
```



Reference Variables

A reference variable is a nickname, or alias, for some other variable

To declare a reference variable, we use the unary operator &

```
int n = 5;    // this declares a variable, n
int & r = n;  // this declares r as a reference to n
```

In this example, r is now a reference to n. (They are both referring to the SAME storage location in memory).



Reference Variables

```
#include <iostream>

using namespace std;

int main()
{
    int x;
    x = 10;

    int &r = x;           // r is now a nickname for x

    cout << '\n';
    cout << "x = " << x << " and r = " << r << '\n';
    cout << "Now we will set r = 20\n\n";

    r = 20;               // changing r will change x. They are
                          // now names for the same storage location

    cout << "x = " << x << " and r = " << r << '\n';
}
```

- While the above code example shows what a reference variable is, you will not likely use it this way!
- In this example, the regular variable and the reference are in the *same scope*, so it seems silly. ("Why do I need to call it r when I can call it x ?")
- So when are references useful? When the two variables are in *different scopes* (this means functions!)



Passing Arguments By Reference

```
#include <iostream>

using namespace std;
// Swap two variables

void swap(int &n1, int &n2)
{
    cout << "\tInside the swap function" << endl;
    cout << "\tBefore swapping n1 is " << n1 << " n2
is " << n2 << endl;
    cout << &n1 << endl;
    cout << &n2 << endl;
    // Swap n1 with n2

    int temp = n1;

    n1 = n2;
    n2 = temp;
    cout << "\tAfter swapping n1 is " << n1 << "
n2 is " << n2 << endl;
}
```

```
int main()

{
    // Declare and initialize variables

    int num1 = 1;
    int num2 = 2;

    cout << "Before invoking the swap
function, num1 is "<< num1 << " and num2 is "
<< num2 << endl;
    // Invoke the swap function to attempt to
    swap two variables

    cout << &num1 << endl;
    cout << &num2 << endl;

    swap(num1, num2);
    cout << "After invoking the swap function,
num1 is " << num1 << " and num2 is " << num2 <<
endl;

    return 0;
}
```



Exercise

Distance

Write a program which calculates the distance between two points $P1(x1, y1)$ and $P2(x2, y2)$.

Input

Four real numbers $x1, y1, x2$ and $y2$ are given in a line.

Output

Print the distance in real number. The output should not contain an absolute error greater than 10^{-4} .

Sample Input

0 0 1 1

Sample Output

1.41421356



Exercise

Triangle

For given two sides of a triangle a and b and the angle C between them, calculate the following properties:

S : Area of the triangle

L : The length of the circumference of the triangle

h : The height of the triangle with side a as a bottom edge

Input

The length of a , the length of b and the angle C are given in integers.

Output

Print S , L and h in a line respectively. The output should not contain an absolute error greater than 10^{-4} .

Sample Input

4 3 90

Sample Output

6.00000000

12.00000000

3.00000000



Exercise

Standard Deviation

You have final scores of an examination for n students. Calculate standard deviation of the scores $s_1, s_2 \dots s_n$. The variance α^2 is defined by

$$\alpha^2 = (\sum_{i=1}^n (s_i - m)^2) / n$$

where m is an average of s_i . The standard deviation of the scores is the square root of their variance.

Input

The input consists of multiple datasets. Each dataset is given in the following format:

n
 $s_1 s_2 \dots s_n$

The input ends with single zero for n .

Output

For each dataset, print the standard deviation in a line. The output should not contain an absolute error greater than 10^{-4} .

Constraints

$$n \leq 1000$$

$$0 \leq s_i \leq 100$$

Sample Input

```
5
70 80 100 90 20
3
80 80 80
0
```

Sample Output

```
27.85677655
0.00000000
```



Exercise

Distance II

Your task is to calculate the distance between two n dimensional vectors $x=\{x_1,x_2,\dots,x_n\}$ and $y=\{y_1,y_2,\dots,y_n\}$. The Minkowski's distance defined below is a metric which is a generalization of both the Manhattan distance and the Euclidean distance.

$$D_{xy} = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}}$$

It can be the Manhattan distance where $p=1$.

$$D_{xy} = |x_1 - y_1| + |x_2 - y_2| + \dots + |x_n - y_n|$$

It can be the Euclidean distance where $p=2$.

$$D_{xy} = \sqrt{(|x_1 - y_1|)^2 + (|x_2 - y_2|)^2 + \dots + (|x_n - y_n|)^2}$$

Also, it can be the Chebyshev distance where $p=\infty$

$$D_{xy} = \max_{i=1}^n (|x_i - y_i|)$$

Input

In the first line, an integer n is given. In the second and third line, $x=\{x_1,x_2,\dots,x_n\}$ and $y=\{y_1,y_2,\dots,y_n\}$ are given respectively. The elements in x and y are given in integers.

Output

Print the distance where $p=1,2,3$ and ∞ in a line respectively. The output should not contain an absolute error greater than 10^{-5} .

Constraints

$$1 \leq n \leq 100, \quad 0 \leq x_i, y_i \leq 1000$$

Sample Input

3

1 2 3

2 0 4

Sample Output

4.000000

2.449490

2.154435

2.000000